

OCI Deployment and Terraform Automation

August 2026

OCI Compute Deployment and Terraform Automation

Ali Youssef

Week 1: OCI Console and Terraform
Full Deployment, Concepts, Troubleshooting, and Remediation Log

Intern	Ali Youssef
Account email	aliyousef.egy@gmail.com
Programme	Ejada Interim Program (2026-intern-program)
Assigned compartment	intern-02-ali-youssef-cmp
Region	Saudi Arabia West (Jeddah), me-jeddah-1
Terraform project path	C:\Users\yinya\Documents\oci-terraform-lab



Contents

1	Executive Summary	1
1.1	Status at a Glance	2
2	Background and Key Concepts	2
2.1	Oracle Cloud Infrastructure Core Building Blocks	2
2.1.1	Tenancy, Region, and Availability Domain	2
2.1.2	Compartment	3
2.1.3	VCN (Virtual Cloud Network)	3
2.1.4	Subnet	3
2.1.5	Internet Gateway (IGW)	3
2.1.6	Route Table	3
2.1.7	Security List	3
2.1.8	Compute Instance	3
2.1.9	Boot Volume and Block Volume	4
2.1.10	File Storage Service (FSS) and NFS	4
2.2	What Terraform Is and Why It Is Used	4
2.3	Terraform Project Structure	4
2.4	Terraform Command Workflow	5
2.5	How Terraform Authenticates to OCI	5
3	Environment Details	6
4	The Lab: One Environment, Two Tasks	6
5	Task 1: Manual OCI Console Deployment	6
5.1	Objective	6
5.2	SSH Key Pair Generation	7
5.3	Virtual Cloud Network (VCN)	7
5.4	Internet Gateway	7
5.5	Route Table	7
5.6	Security List	7
5.7	Public Subnet	8
5.8	Linux Compute Instance	8
5.9	SSH Connectivity Verification	8
5.10	Instance Stopped During Troubleshooting	8
5.11	Remaining Block Volume and File System Steps	8
6	Task 2: Terraform Deployment	9
6.1	Objective	9

6.2	Initial Project Structure	9
6.3	Provider Configuration (provider.tf)	10
6.4	Initial main.tf Configuration	10
6.5	First Deployment Attempt and Failure	10
7	Troubleshooting and Root Cause Analysis	10
7.1	Supervisor's Request	10
7.2	Verification Method	11
7.3	Check 1: Provider Authentication	11
7.4	Check 2: Compartment Placement	11
7.5	Root Cause	11
8	Remediation	11
8.1	Code Fix	12
8.2	Redeployment Result	12
8.3	Duplicate Networking Resources	13
9	Task Requirements Gap Analysis	13
9.1	Task 1 Gap	13
9.2	Task 2 Gap	13
10	Task 2 Extension: Compute, Block Volume, and File Storage	13
10.1	Added Terraform Resources	14
10.2	New Variables (variables.tf)	14
10.3	New Outputs (outputs.tf)	14
10.4	Cost Considerations	14
10.5	Deployment Status: Complete	15
10.6	In-Guest Mounting: Complete	15
10.6.1	Block Volume	16
10.6.2	File System (NFS)	16
10.6.3	Persistence Across Reboots	16
11	Current Status and Outstanding Items	17
12	Glossary of Terms	18
13	Appendix A: Full Terraform Source (Current State)	20
13.1	provider.tf	20
13.2	variables.tf	20
13.3	main.tf	22
13.4	outputs.tf	26

1. Executive Summary

This report documents one laboratory assignment completed through two implementation approaches. Task 1 used the Oracle Cloud Infrastructure (OCI) Console to build the environment manually, while Task 2 reproduced the same environment through Terraform configuration. The report records the underlying concepts, the implementation sequence, the investigation that identified the Terraform failure, the corrective action, and the later extension of the Terraform configuration to cover compute, block storage, and File Storage Service.

The networking and compute components of the manual Console deployment were completed successfully at an early stage. The first Terraform deployment did not succeed, although the failure was not immediately obvious. Each application attempt left the Terraform state containing only a data-source lookup and no managed infrastructure resources. Following a supervisor review, two areas were verified: whether the VCN was being created in the intern's assigned compartment, and whether the authentication parameters in the OCI provider were correct.

The authentication configuration was found to be accurate. The actual problem was the value assigned to `compartment_ocid` in `terraform.tfvars`. It referred to the shared parent compartment, `2026-intern-program`, rather than the assigned leaf compartment, `intern-02-ali-youssef-cmp`. The intern account did not have direct resource-management permission in the parent compartment. The configuration was corrected by resolving the assigned compartment by name within Terraform instead of relying on a manually copied OCID. After this change, `terraform apply` completed successfully.

The Terraform configuration was then extended to include the components that were still missing from Task 2: a compute instance, a block volume, and a File Storage Service file system with an NFS export. The complete configuration was applied successfully. The operating-system work that Terraform did not perform directly, including formatting and mounting the block volume, mounting the NFS export, and configuring both mounts to persist across reboots, was completed manually through SSH. Task 2 is therefore complete from end to end. The only remaining work is the equivalent block-volume and file-system implementation for Task 1 through the OCI Console.

1.1 Status at a Glance

Component	Task 1 (Console)	Task 2 (Terraform)
VCN	Complete: terraform-lab-vcn-AliYousef (10.0.0.0/16)	Complete: tf-lab-vcn-AliYousef (10.1.0.0/16)
Public subnet	Complete: terraform-lab-public-subnet-AliYousef	Complete: tf-lab-public-subnet-AliYousef
Internet gateway and route	Complete	Complete
Security list (SSH and NFS)	SSH complete; NFS rules not yet added	Complete
Linux compute instance	Complete: terraform-lab-linux-AliYousef, currently stopped	Complete: tf-lab-linux-AliYousef, running at 193.122.79.150
Attached block volume	Complete	Complete, formatted, mounted at /mnt/data, and persistent
Mounted file system (NFS)	Complete	Complete, mounted at /mnt/shared, and persistent

Task 2 is fully complete, including the in-guest mounts and persistence configuration described in Sections 10.5 and 10.6.

2. Background and Key Concepts

Both tasks construct the same target architecture. It is therefore useful to define the OCI components and Terraform concepts before examining the two implementations. This section introduces the relevant OCI building blocks, explains the purpose of Terraform, outlines the structure of the project, summarises the command workflow, and describes the OCI authentication process.

2.1 Oracle Cloud Infrastructure Core Building Blocks

Tenancy, Region, and Availability Domain

A tenancy is the root container of an OCI account. Every user, compartment, and resource belongs to one tenancy, which is identified by a unique Oracle Cloud Identifier (OCID). A region is a geographical area containing OCI data centres. This environment uses me-jeddah-1, the Saudi Arabia West region in Jeddah. Within a region, OCI separates resources into availability domains, which are physically isolated locations designed to limit the effect of a failure in one location on the others.

Compartment

A compartment is a logical boundary used to organise resources and apply access-control policies, regardless of the physical location of those resources. Every OCI resource belongs to one compartment. Identity and Access Management (IAM) policies grant permissions at compartment level and may also apply to nested subcompartments. In this internship tenancy, each intern receives a separate leaf compartment, such as `intern-02-ali-youssef-cmp`, under the shared parent compartment `2026-intern-program`. This structure allows an intern to manage resources in the assigned compartment without gaining access to the resources of other interns or to the administrative settings of the parent compartment. This access boundary was central to the Task 2 failure analysed later in the report.

VCN (Virtual Cloud Network)

A Virtual Cloud Network is OCI's software-defined private network. It is comparable to a private network in a physical data centre, but it is implemented virtually within OCI. A VCN is defined by a Classless Inter-Domain Routing (CIDR) range, such as `10.0.0.0/16`, and provides the network boundary within which the associated subnets, gateways, route tables, and security controls operate.

Subnet

A subnet divides the address range of a VCN into a smaller network segment. Instances in a public subnet may receive public IP addresses, provided that the gateway, routing, and security configuration also permit internet access. A private subnet does not allow public IP assignment and is normally used for services that should not be directly accessible from the internet.

Internet Gateway (IGW)

An Internet Gateway connects a VCN to the public internet. The gateway must be attached to the VCN, and a route rule must direct the appropriate traffic towards it. Without both elements, resources in the VCN cannot communicate with external networks, even when the subnet and security rules appear to allow such communication.

Route Table

A route table contains rules that determine where network traffic should be sent according to its destination. The rule used in this lab directs `0.0.0.0/0` to the Internet Gateway. In practical terms, traffic that is not destined for an address within the VCN is forwarded to the public internet.

Security List

A security list is a virtual firewall associated with a subnet. It contains separate ingress and egress rules that determine whether traffic is allowed according to protocol, port, source, and destination. A route table determines where traffic can travel, while a security list determines whether that traffic is permitted. Both configurations must be correct for a connection such as SSH to succeed.

Compute Instance

A compute instance is a virtual machine that combines virtual CPU, memory, storage, and a Virtual Network Interface Card (VNIC). It runs a selected operating-system image on a selected shape, where the shape defines the assigned CPU and memory resources. This project uses Oracle Linux 9 with the `VM.Standard.E2.1.Micro` shape, which is one of OCI's small Always Free eligible shapes.

Boot Volume and Block Volume

Every compute instance receives a boot volume automatically. The boot volume contains the operating system and is required for the instance to function. A block volume is a separate virtual disk that must be created and attached explicitly. It can be detached and attached to another compatible instance without modifying the original operating-system disk. For this reason, the block-volume work in Tasks 1 and 2 is treated as a separate requirement rather than as part of the instance's default storage.

File Storage Service (FSS) and NFS

A block volume is generally attached to one instance at a time, whereas OCI File Storage Service provides shared network storage through the Network File System (NFS) protocol. Access is provided through a mount target, which has its own private IP address in a selected subnet. Multiple instances with network access and suitable security rules can mount the same file system concurrently. FSS is therefore appropriate when several machines need access to the same shared files.

2.2 What Terraform Is and Why It Is Used

Terraform is an open-source Infrastructure as Code tool originally developed by HashiCorp. Instead of creating cloud resources manually through a graphical console, an engineer defines the required infrastructure in configuration files written in HashiCorp Configuration Language (HCL). Terraform then communicates with the cloud provider's API to make the deployed environment match the declared configuration.

Terraform follows a declarative model. The configuration describes the desired result, such as a VCN, subnet, and instance connected in a particular way. It does not require the engineer to define every API action in sequence. Terraform derives the dependency order and decides which resources must be created, updated, or removed. On later runs, it changes only the parts of the environment that no longer match the configuration.

The project uses Terraform for three main reasons:

- **Reproducibility.** The same configuration can create an equivalent environment in another compartment, region, or account without repeating a long series of manual Console actions.
- **Review and version control.** Terraform files are plain text, so changes can be reviewed, compared, tracked, and reverted in the same way as application source code.
- **Drift detection.** The `terraform plan` command identifies differences between the configuration and the actual OCI environment, including changes made manually through the Console.

2.3 Terraform Project Structure

The project at `C:\Users\yinya\Documents\oci-terraform-lab` uses the following Terraform elements:

- **Provider.** A plugin that enables Terraform to communicate with a specific API. This project uses `oracle/oci`, declared in `provider.tf`.
- **Resource.** A configuration block that represents infrastructure Terraform should create and manage, such as `resource "oci_core_vcn" "terraform_vcn" { ... }`.
- **Data source.** A block that reads existing information without creating a resource. The project uses data sources to obtain availability domains, locate an Oracle Linux image, resolve the assigned compartment, and read VNIC details.

- **Variable.** A named input declared in `variables.tf`, with values provided through defaults or `terraform.tfvars`. Variables allow the same configuration to be reused with different account, region, key, shape, or storage settings.
- **Local.** A reusable expression calculated from other values. The remediation stores the compartment OCID returned by the compartment data source in `local.my_compartment_id`.
- **Output.** A value displayed after an application, such as an instance's public IP address, an OCID, or a prepared SSH command.
- **State.** Terraform's record of the real resources it manages, stored in `terraform.tfstate`. State links the deployed OCI resources to the corresponding configuration blocks. The near-empty state file after the first attempt was an important indication that the networking resources had not been created.

2.4 Terraform Command Workflow

The commands used in the project normally follow this order:

- `terraform init` downloads the declared provider plugins and prepares the working directory. It is required when a project is first created and when provider or module requirements change.
- `terraform fmt` reformats the Terraform files into the standard HCL style. It changes presentation only and does not alter the behaviour of the configuration.
- `terraform validate` checks the syntax and internal consistency of the configuration. It operates locally, so a successful validation does not prove that OCI will accept the requested operations.
- `terraform plan` reads the current OCI environment, compares it with the configuration, and displays the proposed additions, changes, and deletions. Saving the plan with `-out=tfplan` ensures that the reviewed plan is the one later applied.
- `terraform apply` executes the selected plan and sends the required create, update, or delete requests to OCI.
- `terraform destroy` removes all resources currently managed through the Terraform state. It provides a controlled way to dismantle the lab environment when it is no longer required.

2.5 How Terraform Authenticates to OCI

OCI API authentication is separate from the password used to access the Console. It uses a public and private RSA key pair. The public key is uploaded to the OCI user profile under Tokens and Keys, while the private key remains on the local machine. OCI calculates a fingerprint for the public key so that requests can identify the key used for signing.

Terraform signs each API request with the private key. OCI verifies the signature by locating the corresponding public key through the supplied fingerprint. The provider therefore requires the tenancy OCID, user OCID, fingerprint, region, and local private-key path. An incorrect tenancy OCID, user OCID, fingerprint, or private key causes the request to fail before OCI evaluates the requested resource operation. This is why the provider authentication values were checked first during the investigation, even though they were later confirmed to be correct.

3. Environment Details

The identifiers below are referenced throughout the report and the Terraform configuration. They were compared directly with the OCI Console during the troubleshooting process.

Username	aliyousef.egy@gmail.com
Identity domain	Ejada-interim-program
Region	me-jeddah-1 (Saudi Arabia West, Jeddah)
Tenancy OCID	ocid1.tenancy.oc1. .aaaaaaats3vpt43eyb7d6djyot4nzy4d7qqe4ajiwr2vnm2rbffcd034nq
User OCID	ocid1.user.oc1. .aaaaaaaaidmdv3zwbjqni6ug4ivi3fool4s32nbozz4nrkd56hipbjltwd5q
API key fingerprint	3e:be:25:77:5c:6b:22:07:38:9f:c7:f9:d8:45:fd:69
Parent compartment	2026-intern-program
Assigned compartment	intern-02-ali-youssef-cmp
SSH key pair, Task 1 instance	C:\Users\yinya\.ssh\oci_vm and oci_vm.pub (Ed25519)
OCI API private key	C:\Users\yinya\.oci\oci_api_key.pem

The OCIDs and fingerprint are identifiers rather than private secrets. Nevertheless, this report should remain internal to the internship programme because the identifiers, when combined with the corresponding private key, form part of the account's authentication configuration.

4. The Lab: One Environment, Two Tasks

This work is one laboratory assignment rather than two unrelated exercises. Both tasks target the same architecture: a VCN, a public subnet, a Linux compute instance, a block volume, and a file system. Task 1 creates these components manually in the OCI Console, which exposes the underlying OCI concepts directly. Task 2 expresses the same architecture as Terraform resources in HCL. Comparing the two implementations demonstrates how each Console field, selection, and action corresponds to a named Infrastructure as Code element.

5. Task 1: Manual OCI Console Deployment

5.1 Objective

The objective of Task 1 was to deploy a VCN, a public subnet, and a Linux compute instance through the OCI Console, then attach a block volume and a file system to the instance.

5.2 SSH Key Pair Generation

An Ed25519 SSH key pair was generated locally in PowerShell. This key pair is used only for SSH access to the Linux instance. It is separate from the OCI API signing key used by Terraform for API authentication.

```
ssh-keygen -t ed25519 -f "$HOME\.ssh\oci_vm" -C "oci-vm"
```

The command created the private key `oci_vm`, which remains local and is never uploaded, and the public key `oci_vm.pub`, which was supplied during instance creation. The instance accepts SSH sessions signed by the corresponding private key.

5.3 Virtual Cloud Network (VCN)

The VCN was created through *Networking > Virtual Cloud Networks > Create VCN* with the following settings:

- Name: `terraform-lab-vcn-AliYousef`
- IPv4 CIDR: `10.0.0.0/16`
- Compartment: `intern-02-ali-youssef-cmp`
- DNS resolution: enabled, with DNS label `terraformlab`

Creating the VCN also created a default route table, default security list, and default DHCP options. These defaults did not provide internet access on their own, so the Internet Gateway, route rule, and ingress rule were still required.

5.4 Internet Gateway

An Internet Gateway named `terraform-lab-igw-AliYousef` was created and enabled. It was attached to the VCN to provide a path between the public subnet and the internet.

5.5 Route Table

A rule was added to the VCN's default route table to send non-local traffic to the Internet Gateway:

- Target type: Internet Gateway
- Destination CIDR: `0.0.0.0/0`
- Target: `terraform-lab-igw-AliYousef`

5.6 Security List

An ingress rule for SSH was added to the VCN's default security list:

- Source: `0.0.0.0/0`. This broad rule was acceptable for the lab, although a specific public IP with a /32 mask is recommended outside a temporary exercise.
- Protocol: TCP, destination port 22.

The default egress rule allowing traffic to `0.0.0.0/0` remained in place. Outbound connections were therefore unrestricted, while inbound access was limited to the configured SSH rule.

5.7 Public Subnet

The public subnet used the following configuration:

- Name: terraform-lab-public-subnet-AliYousef
- IPv4 CIDR: 10.0.1.0/24
- Access type: public subnet, allowing instance VNICs to receive public IP addresses
- Route table: the VCN's default route table
- Security list: the VCN's default security list

5.8 Linux Compute Instance

The Linux instance was created with the following values:

- Name: terraform-lab-linux-AliYousef
- Image: Oracle Linux 9
- Shape: VM.Standard.E2.1.Micro, an Always Free eligible shape
- VCN and subnet: terraform-lab-vcn-AliYousef and terraform-lab-public-subnet-AliYousef
- Public IPv4 address: assigned
- SSH key: oci_vm.pub

Oracle Linux images use the opc account by default. Authentication is performed with the uploaded public key rather than a password.

5.9 SSH Connectivity Verification

The following command was used to connect to the instance:

```
ssh -i "$HOME/.ssh/oci_vm" opc@<PUBLIC_IP>
```

5.10 Instance Stopped During Troubleshooting

While the Task 2 failures were being investigated, the manual instance was stopped through the OCI Console using a graceful shutdown. The force-stop option remained disabled. Stopping the instance paused billing for the compute shape without deleting the instance or its boot volume. The boot volume remained stored, and the instance continued to count towards service limits. The instance was not terminated, since termination would have permanently deleted it. At the time of writing, the instance remains stopped and can be started again when needed.

5.11 Remaining Block Volume and File System Steps

Task 1 also requires a separate block volume and an FSS file system. The procedure was documented but had not yet been executed in the Console:

1. Open *Storage > Block Storage > Block Volumes* and create a block volume in intern-02-ali-youssef-cmp, in the same availability domain as the instance. The planned example is a 50 GB volume named lab1-block-volume-AliYousef.
2. Open terraform-lab-linux-AliYousef, select *Attached Block Volumes*, attach the new volume

using the paravirtualised attachment type, and confirm the attachment.

3. Start the instance and connect through SSH. Use `lsblk` to identify the new disk, then format and mount it with commands such as `sudo mkfs -t ext4 /dev/sdb`, `sudo mkdir /mnt/data`, and `sudo mount /dev/sdb /mnt/data`.
4. Open *Storage > File Storage > File Systems* and create a file system in the same compartment. Create a mount target in `terraform-lab-public-subnet-AliYousef` during the same process.
5. Add NFS ingress rules to the VCN's default security list. The source should be `10.0.0.0/16`, with TCP and UDP port 111 and TCP and UDP ports 2048 to 2050.
6. Mount the export using the mount command shown on the file-system page, for example `sudo mkdir /mnt/shared` followed by `sudo mount <MOUNT_TARGET_IP>:/<export-path> /mnt/shared`.

These Console steps correspond directly to the Terraform storage resources added for Task 2 in Section 10.

6. Task 2: Terraform Deployment

6.1 Objective

Task 2 reproduced the Task 1 architecture as Terraform configuration so that the environment would be repeatable, reviewable, and version-controlled rather than dependent on a sequence of manual Console actions.

6.2 Initial Project Structure

```
oci-terraform-lab/
  .terraform/           (created by terraform init; downloaded provider plugin)
  .terraform.lock.hcl   (records exact provider version and checksums)
  .gitignore            (keeps state and secrets out of version control)
  provider.tf           (provider and authentication; Section 2.5)
  variables.tf          (input variable declarations)
  terraform.tfvars      (values supplied to the variables)
  main.tf               (managed infrastructure resources)
  outputs.tf            (values displayed after apply)
```

6.3 Provider Configuration (provider.tf)

The provider configuration declares the oracle/oci plugin and supplies the five values required to authenticate each API request.

```
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    oci = {
      source  = "oracle/oci"
      version = "~> 7.0"
    }
  }
}

provider "oci" {
  tenancy_ocid    = var.tenancy_ocid
  user_ocid       = var.user_ocid
  fingerprint     = var.fingerprint
  private_key_path = var.private_key_path
  region          = var.region
}
```

6.4 Initial main.tf Configuration

The first version of `main.tf` defined five networking resources: a VCN, Internet Gateway, public route table, security list, and public subnet. Each resource corresponded to one of the manual Console steps in Task 1. All five resources used `var.compartment_ocid`, whose value had been copied manually into `terraform.tfvars`.

6.5 First Deployment Attempt and Failure

The first terraform apply did not present an immediately useful explanation of the failure. The resulting state file contained only the availability-domain data-source lookup. None of the five networking resources had been recorded or created. Because the intern account had also reported difficulty viewing the compartment list in the OCI Console, the initial investigation considered IAM permissions as a likely cause.

7. Troubleshooting and Root Cause Analysis

7.1 Supervisor's Request

After the continuing failure was reported, the supervisor requested two checks:

- Confirm that the VCN was being created in the intern's assigned compartment.
- Confirm that the OCI provider authentication block and every parameter within it were correct.

7.2 Verification Method

The checks were performed by comparing the OCI Console in Chrome with the Terraform project in Visual Studio Code. The values in `terraform.tfvars`, `provider.tf`, and `variables.tf` were inspected alongside the live account information. This direct comparison avoided relying on assumptions or on another reading of the code alone.

7.3 Check 1: Provider Authentication

Each authentication value was compared character by character with the OCI Console's configuration-file preview under Tokens and Keys. The preview showed the exact user OCID, tenancy OCID, fingerprint, and region associated with the uploaded API key.

Field	OCI Console value	terraform.tfvars value
Region	me-jeddah-1	me-jeddah-1, match
User OCID	Ending ...hipbjltwd5q	Ending ...hipbjltwd5q, match
Tenancy OCID	Ending ...rbffcd034nq	Ending ...rbffcd034nq, match
Fingerprint	3e:be:25:...:fd:69	3e:be:25:...:fd:69, match

The comparison confirmed that the provider authentication block was correct and was not responsible for the failed deployment.

7.4 Check 2: Compartment Placement

The compartment check required a more detailed investigation. Opening the parent compartment 2026-intern-program through *Identity and Security > Compartments* produced the message "Authorization failed or requested resource not found". This showed that the intern account could not open the administrative page of the parent compartment.

A global resource search confirmed that the resources created manually in Task 1, including the VCN, subnet, and Internet Gateway, were located in intern-02-ali-youssef-cmp. The VCN details page also displayed this compartment explicitly.

The value of `compartment_ocid` in `terraform.tfvars` was then inspected. The full value was 83 characters long, and its prefix matched the OCID of 2026-intern-program, the parent compartment that had returned the authorisation error. It did not refer to the intern's assigned leaf compartment.

7.5 Root Cause

The `compartment_ocid` variable pointed to 2026-intern-program instead of intern-02-ali-youssef-cmp. The intern account did not have direct create or manage permission in the parent compartment. Since every resource in `main.tf` inherited this value through `var.compartment_ocid`, OCI's IAM layer rejected every resource-creation request. This finding was consistent with both the nearly empty Terraform state and the earlier authorisation error in the Console.

8. Remediation

8.1 Code Fix

Instead of manually copying another compartment OCID, the corrected configuration resolves the assigned compartment by name during each plan or application. The OCI identity data source searches the tenancy subtree and returns the active compartment named `intern-02-ali-youssef-cmp`. This approach requires list access, which the account already has because the OCI Console compartment selector can display the intern compartments.

The following blocks were added to `main.tf`:

```
data "oci_identity_compartments" "mine" {
  compartment_id           = var.tenancy_ocid
  compartment_id_in_subtree = true
  name                     = "intern-02-ali-youssef-cmp"
  state                    = "ACTIVE"
}

locals {
  my_compartment_id = data.oci_identity_compartments.mine.compartments[0].id
}
```

For the five networking resources, `compartment_id = var.compartment_ocid` was replaced with `compartment_id = local.my_compartment_id`. The local value is calculated once and reused by the VCN, Internet Gateway, route table, security list, and subnet.

8.2 Redeployment Result

After the correction, the Terraform workflow was executed in sequence:

```
terraform init -upgrade    -> Terraform has been successfully initialized!
terraform validate         -> Success! The configuration is valid.
terraform plan -out=tfplan -> Plan: 5 to add, 0 to change, 0 to destroy.
terraform apply tfplan     -> Apply complete! Resources: 5 added, 0 changed, 0
                             destroyed.
```

All five networking resources were created in the correct compartment. The resolved compartment OCID ended in `...ybrfsq`, which differed from the previously hard-coded parent OCID.

- `oci_core_vcn.terraform_vcn: tf-lab-vcn-AliYousef, 10.1.0.0/16`
- `oci_core_internet_gateway.terraform_igw: tf-lab-igw-AliYousef`
- `oci_core_route_table.public_route_table: tf-lab-public-route-table-AliYousef`
- `oci_core_security_list.public_security_list: tf-lab-public-security-list-AliYousef`
- `oci_core_subnet.public_subnet: tf-lab-public-subnet-AliYousef, 10.1.1.0/24`

8.3 Duplicate Networking Resources

The manual VCN for Task 1 uses `10.0.0.0/16`, while the Terraform VCN for Task 2 uses `10.1.0.0/16`. Both now exist in `intern-02-ali-youssef-cmp`. The compartment therefore contains two parallel sets of VCNs, subnets, Internet Gateways, route tables, and security lists, one set for each task. This is expected for the two lab deliverables, but the resources must be distinguished carefully during review and later cleanup.

9. Task Requirements Gap Analysis

After the Terraform networking deployment succeeded, the original assignment requirements were compared with the resources that actually existed. This review identified one outstanding gap in each task.

9.1 Task 1 Gap

Task 1 required both an attached block volume and a file system. At that stage, the manual instance had only its automatically created boot volume. No separate block volume or File Storage Service file system had been created.

9.2 Task 2 Gap

At the same stage, `main.tf` contained networking resources only. It did not contain an `oci_core_instance` resource, so Terraform had not created a virtual machine, block volume, or file system. Section 10 describes how this gap was closed, while Section 5.11 records the remaining Console procedure for Task 1.

10. Task 2 Extension: Compute, Block Volume, and File Storage

To make Task 2 equivalent in scope to Task 1, `main.tf`, `variables.tf`, and `outputs.tf` were extended directly. Access to `C:\Users\yinya\Documents\oci-terraform-lab` was available during this work.

10.1 Added Terraform Resources

The extension introduced the following resources and data sources:

- data "oci_core_images" "oracle_linux" selects the newest Oracle Linux 9 image compatible with the configured shape, avoiding a hard-coded image OCID.
- resource "oci_core_instance" "linux_instance" creates tf-lab-linux-AliYousef, assigns a public IP address, and injects the SSH public key through instance metadata.
- resource "oci_core_volume" "data_volume" creates the 50 GB block volume tf-lab-block-volume-AliYousef in the same availability domain as the instance.
- resource "oci_core_volume_attachment" "data_volume_attachment" attaches the block volume using the paravirtualised attachment type.
- resource "oci_file_storage_file_system" "shared_fs" creates tf-lab-fs-AliYousef.
- resource "oci_file_storage_mount_target" "shared_mount_target" creates the mount target in the Terraform-managed public subnet.
- resource "oci_file_storage_export" "shared_export" exposes the file system at /tf-lab-export-aliyousef.
- Four additional ingress rules allow TCP and UDP port 111 and TCP and UDP ports 2048 to 2050. Their source is limited to 10.1.0.0/16, the VCN's own CIDR range.
- data "oci_core_vnic_attachments" and data "oci_core_vnic" read the instance VNIC and public IP address for use in the Terraform outputs.

10.2 New Variables (variables.tf)

Variable	Default	Purpose
instance_shape	VM.Standard.E2.1.Micro	Uses the same Always Free eligible shape as the manual Task 1 instance.
ssh_public_key_path	C:/Users/yinya/.ssh/oci_vm.pub	Reuses the SSH key pair generated for Task 1.
block_volume_size_in_gbs	50	May be overridden in terraform.tfvars when another volume size is required.

10.3 New Outputs (outputs.tf)

- instance_id, instance_public_ip, and ssh_command identify the instance and provide the connection command.
- block_volume_id provides the OCID of the attached block volume.
- file_system_export_path provides the NFS export path.
- mount_target_private_ip_ids provides the private-IP object OCIDs required to locate the mount target's usable IP address.

10.4 Cost Considerations

The 50 GB block volume falls within OCI's Always Free allowance of up to 200 GB of block storage. File Storage Service is not included in the Always Free tier and is billed according to stored capacity over time. Because this is a shared internship tenancy, the supervisor should confirm whether the FSS resource should remain provisioned after the lab has been reviewed.

10.5 Deployment Status: Complete

The extended Terraform configuration was applied successfully. The plan contained six new resources, comprising the instance, block volume, volume attachment, file system, mount target, and export. It also contained one in-place change to add the four NFS rules to the security list.

```
Plan: 6 to add, 1 to change, 0 to destroy.
...
oci_core_security_list.public_security_list: Modifications complete after 1s
oci_file_storage_file_system.shared_fs: Creation complete after 2s
oci_core_volume.data_volume: Creation complete after 14s
oci_file_storage_mount_target.shared_mount_target: Creation complete after 15s
oci_file_storage_export.shared_export: Creation complete after 5s
oci_core_instance.linux_instance: Creation complete after 35s
oci_core_volume_attachment.data_volume_attachment: Creation complete after 24s

Apply complete! Resources: 6 added, 1 changed, 0 destroyed.
```

The final values returned by terraform output were:

```
instance_id = ocid1.instance.oc1.me-jeddah-1.
  avgkljrvjakbncwx5n5zxd54l7m36yyynqh74esmfkg7adro75g2ry34wa
instance_public_ip = 193.122.79.150
ssh_command = ssh -i C:/Users/yinya/.ssh/oci_vm opc@193.122.79.150
block_volume_id = ocid1.volume.oc1.me-jeddah-1.
  abvgkljr6moz5t4zc3l3kqkk64yytu42aacdhproxfh6lr7n6ksknfjz74iq
file_system_export_path = /tf-lab-export-aliyousef
mount_target_private_ip_ids = ocid1.privateip.oc1.me-jeddah-1.
  aaaaaaabnr6in7liemaadpkk5qz3qysb3a2emktentqxdvrbtq67vrwfdna
```

Task 2 now includes the network, compute instance, attached block volume, File Storage Service file system, mount target, and NFS export. Every component was created through Terraform and matches the required scope. The instance is running at 193.122.79.150 and is accessible with the SSH command shown above.

10.6 In-Guest Mounting: Complete

Terraform created and attached the storage resources at OCI level, but it did not format or mount them inside Oracle Linux. These operating-system steps were completed manually after connecting through SSH.

Block Volume

The `lsblk` output showed two disks. The new block volume appeared as `/dev/sda`, a blank 50 GB device with no partitions. The boot volume appeared as `/dev/sdb`, a 46.6 GB device containing the EFI, boot, and LVM root partitions. The devices were checked carefully before formatting because selecting the boot disk would have destroyed the operating system.

```
$ sudo mkfs -t ext4 /dev/sda
Filesystem UUID: 407adadb-af7a-43b8-9024-ee009281cc2b
$ sudo mkdir -p /mnt/data
$ sudo mount /dev/sda /mnt/data
$ df -h /mnt/data
/dev/sda    49G   24K   47G   1% /mnt/data
```

File System (NFS)

The mount target's private IP address was `10.1.1.249`. It was found through *Console > Storage > File Storage > Mount Targets > tf-lab-mount-target-AliYousef*. The NFS utilities were installed, and the export was mounted as follows:

```
$ sudo dnf install -y nfs-utils
$ sudo mkdir -p /mnt/shared
$ sudo mount 10.1.1.249:/tf-lab-export-aliyousef /mnt/shared
$ df -h /mnt/shared
10.1.1.249:/tf-lab-export-aliyousef  8.0E   0   8.0E   0% /mnt/shared
```

The reported capacity of `8.0E` is expected. FSS is elastic shared network storage rather than a fixed-size disk, so `df` reports a very large nominal capacity instead of a provisioned disk size.

Persistence Across Reboots

Neither mount persists after a reboot unless it is defined in `/etc/fstab`. The following entries were added. The block volume uses its UUID instead of a device name because Linux device names can change between boots.

```
UUID=407adadb-af7a-43b8-9024-ee009281cc2b /mnt/data    ext4 defaults,nofail
0 2
10.1.1.249:/tf-lab-export-aliyousef /mnt/shared nfs defaults,nofail,
_netdev 0 0
```

The `nofail` option allows the instance to boot even when a volume or network mount is temporarily unavailable. The `_netdev` option indicates that the NFS mount depends on network availability. The entries were verified with `sudo mount -a`, which applies the `fstab` configuration without requiring a reboot.

11. Current Status and Outstanding Items

Component	Task 1 (Console)	Task 2 (Terraform)
VCN, subnet, and instance	Complete	Complete
Block volume attached at OCI level	Documented, not yet executed	Complete
Block volume formatted and mounted in guest	Documented, not yet executed	Complete at <code>/mnt/data</code> , with 47 GB available
File system, mount target, and export	Documented, not yet executed	Complete
File system mounted in guest	Documented, not yet executed	Complete at <code>/mnt/shared</code> over NFS
Mount persistence across re-boot	Documented, not yet executed	Complete through <code>/etc/fstab</code> entries
Networking	Complete	Complete and applied successfully
Compute instance	Complete, currently stopped	Complete and running at 193.122.79.150

Task 2 is complete from end to end, including the operating-system mounts and reboot persistence. The only unfinished assignment component is the equivalent block-volume and file-system work for Task 1.

The remaining follow-up actions are:

- Complete the Task 1 Console procedure for the block volume and file system described in Section 5.11.
- Decide whether both the manual and Terraform-created VCN environments should remain, or whether one should be removed after the tasks have been reviewed and graded.
- Confirm whether the continuing File Storage Service charge is acceptable or whether the FSS resources should be destroyed after review.
- Restrict the SSH and NFS ingress sources further, such as limiting SSH to a specific public /32 address, if the environment remains active beyond the lab.

12. Glossary of Terms

OCID	Oracle Cloud Identifier, the globally unique identifier OCI assigns to each resource. It is prefixed by the resource type, such as <code>ocid1.compartment.oc1..</code> or <code>ocid1.instance.oc1..</code>
Tenancy	The root container of an OCI account. All compartments, identities, and resources exist within it.
Compartment	A logical grouping used to organise resources and define IAM permissions independently of physical location.
Region / AD	A region is a geographical OCI location, such as <code>me-jeddah-1</code> . An availability domain is a fault-isolated data-centre location within a region.
VCN	Virtual Cloud Network, OCI's software-defined private network, identified by one or more CIDR ranges.
CIDR	Classless Inter-Domain Routing, the notation used to describe an IP-address range, such as <code>10.0.0.0/16</code> .
Subnet	A smaller network segment within a VCN. Public subnets allow public IP assignment, while private subnets do not.
Internet Gateway (IGW)	A virtual gateway that connects a VCN to the public internet.
Route Table	A collection of rules that determines where outbound traffic is sent according to destination address.
Security List	A virtual firewall associated with a subnet that controls traffic by source, destination, protocol, and port.
Shape	A predefined allocation of CPU and memory for a compute instance, such as <code>VM.Standard.E2.1.Micro</code> .
Boot Volume	The disk created automatically with a compute instance and used to store its operating system.
Block Volume	An additional virtual disk that is created separately and attached to an instance for extra storage.
File Storage Service (FSS)	OCI's managed NFS storage service, which can be shared by multiple instances through a mount target.
Mount Target	The network endpoint, including a private IP address, through which instances access an FSS file system.
IAM	Identity and Access Management, the OCI framework of identities, groups, and policies that determines who may perform specific operations in specific compartments.
Terraform	An open-source Infrastructure as Code tool that creates and manages cloud resources from declarative configuration.
HCL	HashiCorp Configuration Language, the language used to write Terraform configuration files.
Provider	A Terraform plugin that enables communication with a particular platform API, such as <code>oracle/oci</code> .
Resource	A Terraform block that defines infrastructure to be created and managed.
Data source	A Terraform block that reads existing information without creating a managed resource.
Variable / Local / Output	A variable is a named input, a local is a calculated reusable value, and an output is a value displayed after an application.

State	Terraform's record, stored in <code>terraform.tfstate</code> , of the resources it manages and their relationship to the configuration.
Plan / Apply	<code>plan</code> previews proposed infrastructure changes, while <code>apply</code> executes those changes.
Fingerprint	A hash of an API public key that OCI uses to identify the key associated with a signed request.

13. Appendix A: Full Terraform Source (Current State)

The following listings reproduce the current Terraform source included in the research record.

13.1 provider.tf

```
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    oci = {
      source  = "oracle/oci"
      version = "~> 7.0"
    }
  }
}

provider "oci" {
  tenancy_ocid      = var.tenancy_ocid
  user_ocid         = var.user_ocid
  fingerprint       = var.fingerprint
  private_key_path  = var.private_key_path
  region            = var.region
}
```

13.2 variables.tf

```
variable "tenancy_ocid" {
  description = "OCI tenancy OCID"
  type        = string
  sensitive   = true
}

variable "user_ocid" {
  description = "OCI user OCID"
  type        = string
  sensitive   = true
}

variable "fingerprint" {
  description = "OCI API key fingerprint"
  type        = string
  sensitive   = true
}

variable "private_key_path" {
  description = "Path to the OCI API private key"
  type        = string
  sensitive   = true
}
```

```
variable "region" {
  description = "OCI region"
  type        = string
  default     = "me-jeddah-1"
}

variable "compartment_ocid" {
  description = "OCID of the compartment where resources will be created"
  type        = string
}

variable "instance_shape" {
  description = "OCI Compute shape for the Linux instance"
  type        = string
  default     = "VM.Standard.E2.1.Micro"
}

variable "ssh_public_key_path" {
  description = "Local path to the SSH public key uploaded to the instance"
  type        = string
  default     = "C:/Users/yinya/.ssh/oci_vm.pub"
}

variable "block_volume_size_in_gbs" {
  description = "Size in GB of the extra block volume attached to the instance"
  type        = number
  default     = 50
}
```


13.3 main.tf

```
data "oci_identity_availability_domains" "ads" {
  compartment_id = var.tenancy_ocid
}

data "oci_identity_compartments" "mine" {
  compartment_id           = var.tenancy_ocid
  compartment_id_in_subtree = true
  name                     = "intern-02-ali-youssef-cmp"
  state                    = "ACTIVE"
}

locals {
  my_compartment_id = data.oci_identity_compartments.mine.compartments[0].id
}

resource "oci_core_vcn" "terraform_vcn" {
  compartment_id = local.my_compartment_id

  display_name = "tf-lab-vcn-AliYousef"
  cidr_blocks  = ["10.1.0.0/16"]
  dns_label    = "tflabvcn"
}

resource "oci_core_internet_gateway" "terraform_igw" {
  compartment_id = local.my_compartment_id
  vcn_id         = oci_core_vcn.terraform_vcn.id

  display_name = "tf-lab-igw-AliYousef"
  enabled      = true
}

resource "oci_core_route_table" "public_route_table" {
  compartment_id = local.my_compartment_id
  vcn_id         = oci_core_vcn.terraform_vcn.id

  display_name = "tf-lab-public-route-table-AliYousef"

  route_rules {
    destination          = "0.0.0.0/0"
    destination_type     = "CIDR_BLOCK"
    network_entity_id    = oci_core_internet_gateway.terraform_igw.id
  }
}

resource "oci_core_security_list" "public_security_list" {
  compartment_id = local.my_compartment_id
}
```

```

vcn_id          = oci_core_vcn.terraform_vcn.id

display_name = "tf-lab-public-security-list-AliYousef"

egress_security_rules {
  destination      = "0.0.0.0/0"
  destination_type = "CIDR_BLOCK"
  protocol         = "all"
  stateless        = false
}

ingress_security_rules {
  source      = "0.0.0.0/0"
  source_type = "CIDR_BLOCK"
  protocol    = "6"
  stateless   = false
  description = "Allow SSH"

  tcp_options {
    min = 22
    max = 22
  }
}

ingress_security_rules {
  source      = "10.1.0.0/16"
  source_type = "CIDR_BLOCK"
  protocol    = "6"
  stateless   = false
  description = "NFS TCP from within the VCN"

  tcp_options {
    min = 111
    max = 111
  }
}

ingress_security_rules {
  source      = "10.1.0.0/16"
  source_type = "CIDR_BLOCK"
  protocol    = "17"
  stateless   = false
  description = "NFS UDP from within the VCN"

  udp_options {
    min = 111
    max = 111
  }
}

ingress_security_rules {
  source      = "10.1.0.0/16"
  source_type = "CIDR_BLOCK"

```

```

    protocol      = "6"
    stateless     = false
    description   = "NFS/mount TCP from within the VCN"

    tcp_options {
      min = 2048
      max = 2050
    }
  }

  ingress_security_rules {
    source      = "10.1.0.0/16"
    source_type = "CIDR_BLOCK"
    protocol    = "17"
    stateless   = false
    description = "NFS/mount UDP from within the VCN"

    udp_options {
      min = 2048
      max = 2050
    }
  }
}

resource "oci_core_subnet" "public_subnet" {
  compartment_id = local.my_compartment_id
  vcn_id         = oci_core_vcn.terraform_vcn.id

  display_name = "tf-lab-public-subnet-AliYousef"
  cidr_block   = "10.1.1.0/24"
  dns_label    = "public"

  route_table_id    = oci_core_route_table.public_route_table.id
  security_list_ids = [oci_core_security_list.public_security_list.id]

  prohibit_public_ip_on_vnic = false
}

data "oci_core_images" "oracle_linux" {
  compartment_id      = local.my_compartment_id
  operating_system     = "Oracle Linux"
  operating_system_version = "9"
  shape               = var.instance_shape
  sort_by             = "TIMECREATED"
  sort_order          = "DESC"
}

resource "oci_core_instance" "linux_instance" {
  compartment_id      = local.my_compartment_id
  availability_domain = data.oci_identity_availability_domains.ads.
    availability_domains[0].name

```

```

display_name = "tf-lab-linux-AliYousef"
shape        = var.instance_shape

create_vnic_details {
  subnet_id      = oci_core_subnet.public_subnet.id
  assign_public_ip = true
  display_name   = "tf-lab-linux-vnic-AliYousef"
  hostname_label = "tflablinux"
}

source_details {
  source_type = "image"
  source_id   = data.oci_core_images.oracle_linux.images[0].id
}

metadata = {
  ssh_authorized_keys = file(var.ssh_public_key_path)
}

preserve_boot_volume = false
}

resource "oci_core_volume" "data_volume" {
  compartment_id      = local.my_compartment_id
  availability_domain = data.oci_identity_availability_domains.ads.
    availability_domains[0].name

  display_name = "tf-lab-block-volume-AliYousef"
  size_in_gbs  = var.block_volume_size_in_gbs
}

resource "oci_core_volume_attachment" "data_volume_attachment" {
  attachment_type = "paravirtualized"
  instance_id     = oci_core_instance.linux_instance.id
  volume_id       = oci_core_volume.data_volume.id
  display_name    = "tf-lab-block-volume-attachment-AliYousef"
}

resource "oci_file_storage_file_system" "shared_fs" {
  compartment_id      = local.my_compartment_id
  availability_domain = data.oci_identity_availability_domains.ads.
    availability_domains[0].name

  display_name = "tf-lab-fs-AliYousef"
}

resource "oci_file_storage_mount_target" "shared_mount_target" {
  compartment_id      = local.my_compartment_id
  availability_domain = data.oci_identity_availability_domains.ads.
    availability_domains[0].name

```

```

    subnet_id          = oci_core_subnet.public_subnet.id

    display_name = "tf-lab-mount-target-AliYousef"
}

resource "oci_file_storage_export" "shared_export" {
    export_set_id = oci_file_storage_mount_target.shared_mount_target.export_set_id
    file_system_id = oci_file_storage_file_system.shared_fs.id
    path          = "/tf-lab-export-aliyousef"
}

data "oci_core_vnic_attachments" "linux_instance" {
    compartment_id = local.my_compartment_id
    instance_id    = oci_core_instance.linux_instance.id
}

data "oci_core_vnic" "linux_instance" {
    vnic_id = data.oci_core_vnic_attachments.linux_instance.vnic_attachments[0].
        vnic_id
}

```

13.4 outputs.tf

```

output "availability_domain_names" {
    description = "Availability domains available in the tenancy"
    value = [
        for ad in data.oci_identity_availability_domains.ads.availability_domains :
        ad.name
    ]
}

output "terraform_vcn_id" {
    description = "OCID of the Terraform-created VCN"
    value      = oci_core_vcn.terraform_vcn.id
}

output "terraform_subnet_id" {
    description = "OCID of the Terraform-created public subnet"
    value      = oci_core_subnet.public_subnet.id
}

output "terraform_internet_gateway_id" {
    description = "OCID of the Terraform-created Internet Gateway"
    value      = oci_core_internet_gateway.terraform_igw.id
}

output "instance_id" {
    description = "OCID of the Terraform-created Linux instance"
    value      = oci_core_instance.linux_instance.id
}

```

```
output "instance_public_ip" {
  description = "Public IPv4 address of the Terraform-created Linux instance"
  value       = data.oci_core_vnic.linux_instance.public_ip_address
}

output "ssh_command" {
  description = "Example SSH command to reach the instance"
  value       = "ssh -i C:/Users/yinya/.ssh/oci_vm opc@${data.oci_core_vnic.
    linux_instance.public_ip_address}"
}

output "block_volume_id" {
  description = "OCID of the extra block volume attached to the instance"
  value       = oci_core_volume.data_volume.id
}

output "file_system_export_path" {
  description = "NFS export path for the file system"
  value       = oci_file_storage_export.shared_export.path
}

output "mount_target_private_ip_ids" {
  description = "OCID(s) of the mount target's private IP object(s). Look these up
    in the console (Networking > Private IPs) or via 'oci network private-ip get' to
    find the actual IP address to mount from."
  value       = oci_file_storage_mount_target.shared_mount_target.private_ip_ids
}
```

Windows PowerShell

Copyright (C) Microsoft Corporation. All rights reserved.

Loading personal and system profiles took 1813ms.

(base) PS C:\WINDOWS\system32> ssh -i "\$HOME\.ssh\oci_vm" opc@79.72.6.225

The authenticity of host '79.72.6.225 (79.72.6.225)' can't be established.

ED25519 key fingerprint is SHA256:OYMPBskTbekg3PYKuUYRbNfcYdJRpdOPeZr1R9MPkdw.

This key is not known by any other names.

Are you sure you want to continue connecting (yes/no/[fingerprint])? y

Please type 'yes', 'no' or the fingerprint: yes

Warning: Permanently added '79.72.6.225' (ED25519) to the list of known hosts.

[opc@terraform-lab-vnic-aliyousef ~]\$ hostname

cat /etc/os-release

ip addr

terraform-lab-vnic-aliyousef

NAME="Oracle Linux Server"

VERSION="9.8"

ID="ol"

ID_LIKE="fedora"

VARIANT="Server"

VARIANT_ID="server"

VERSION_ID="9.8"

PLATFORM_ID="platform:el9"

PRETTY_NAME="Oracle Linux Server 9.8"

ANSI_COLOR="0;31"

CPE_NAME="cpe:/o:oracle:linux:9:8:server"

HOME_URL="https://linux.oracle.com/"

BUG_REPORT_URL="https://github.com/oracle/oracle-linux"

ORACLE_BUGZILLA_PRODUCT="Oracle Linux 9"

ORACLE_BUGZILLA_PRODUCT_VERSION=9.8

ORACLE_SUPPORT_PRODUCT="Oracle Linux"

ORACLE_SUPPORT_PRODUCT_VERSION=9.8

1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000

link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00

Cloud Sign In

Login

Instances | Oracle Cloud Infrastr

+

Ask Gemini

cloud.oracle.com/compute/instances/ocid1.instance.oc1.me-jeddah-1.anvgkljrvjakbnicab32viarlzof35jr3zzur3v66pptjtwkzotf5rl2kw4a/details?region=me-jedda...

ai guc everyday GLC ppt Thesis datalsBeauty professional typing learning investing tech personal social cars art

Cloud

Search resources, services, documentation, and Marketplace

Saudi Arabia West (Jeddah)

Compute

Compute

Overview

Instances

Instance Maintenance

Dedicated Virtual Machine Hosts

Instance Configurations

Instance Pools

Cluster Networks

Compute Clusters

← Instances

terraform-lab-linux-AliYousefRunning

ActionsStart

DetailsNetworkingStorageSecurityManagementOS ManagementMonitoringWork requestsTags

General information

Availability domain	AD-1
Fault domain	FD-2
Region	me-jeddah-1
OCID	...jeddah-1.anvgkljrvjakbnicab32viarlzof35jr3zzur3v66pptjtwkzotf5rl2kw4aCopy
Launched	Aug 05, 2026, 07:52:59 UTC

Terms of Use and Privacy

Cookie Preferences

Copyright © 2026, Oracle and/or its affiliates. All rights reserved.

Compute

< Compute

Overview

Instances

Instance Maintenance

Dedicated Virtual Machine
Hosts

Instance Configurations

Instance Pools

Cluster Networks

Compute Clusters



Details

Networking

Storage

Security

Management

OS Management

Monitoring

Work requests

Tags

Applied filters

Namespace `oci_computeagent`Date `Aug 05, 2026, 07:00:10 UTC - Au...`

CPU Utilization

...

Activity level from CPU. Expressed as a percentage of total time (busy and idle) versus idle time. A typical alarm threshold is 90 percent.

Applied filters

Interval `Auto`Statistic `Mean`

Memory Utilization

...

Space currently in use. Measured by pages. Expressed as a percentage of used pages versus unused pages. A typical alarm threshold is 85 percent.

Applied filters

Interval `Auto`Statistic `Mean`

Cloud Sign In

Login

Instances | Oracle Cloud Infrastr

Ask Gemini

cloud.oracle.com/compute/instances/ocid1.instance.oc1.me-jeddah-1.anvgkljrvjakbnicab32viarlzof35jr3zzur3v66pptjtwkzotf5rl2kw4a/details?region=me-jedda...

ai, guc, everyday, GLC, ppt, Thesis, datalsBeauty, professional, typing, learning, investing, tech, personal, social, cars, art

Cloud

Search resources, services, documentation, and Marketplace

Saudi Arabia West (Jeddah)

Compute

Compute

Overview

Instances

Instance Maintenance

Dedicated Virtual Machine Hosts

Instance Configurations

Instance Pools

Cluster Networks

Compute Clusters

Details

Networking

Storage

Security

Management

OS Management

Monitoring

Work requests

Tags

pair that was used to create the instance.

Public IP address

79.72.6.225

Copy

Username

opc

Instance details

Virtual cloud network

terraform-lab-vcn-AliYousef

Maintenance reboot

—

Launch mode

PARAVIRTUALIZED

Instance metadata service

Version 2 only

Edit

The instance metadata service provides metadata about the instance. Applications can use this metadata to bootstrap or do other tasks.

Terms of Use and Privacy

Cookie Preferences

Copyright © 2026, Oracle and/or its affiliates. All rights reserved.